

Types of Tests in JUnit

JUnit supports **several types** of **test methods**: `@Test`, `@ParameterizedTest`, and `@RepeatedTest`

 **by Rodrigo Martins Pagliares**

Last update: 11/27/2025



Regular Tests vs. Parameterized vs. Repeated

Regular Tests

Regular tests use `@Test` and run **once** per invocation.

Parameterized Tests

Parameterized tests use `@ParameterizedTest` and run **multiple times, once for each argument set**.

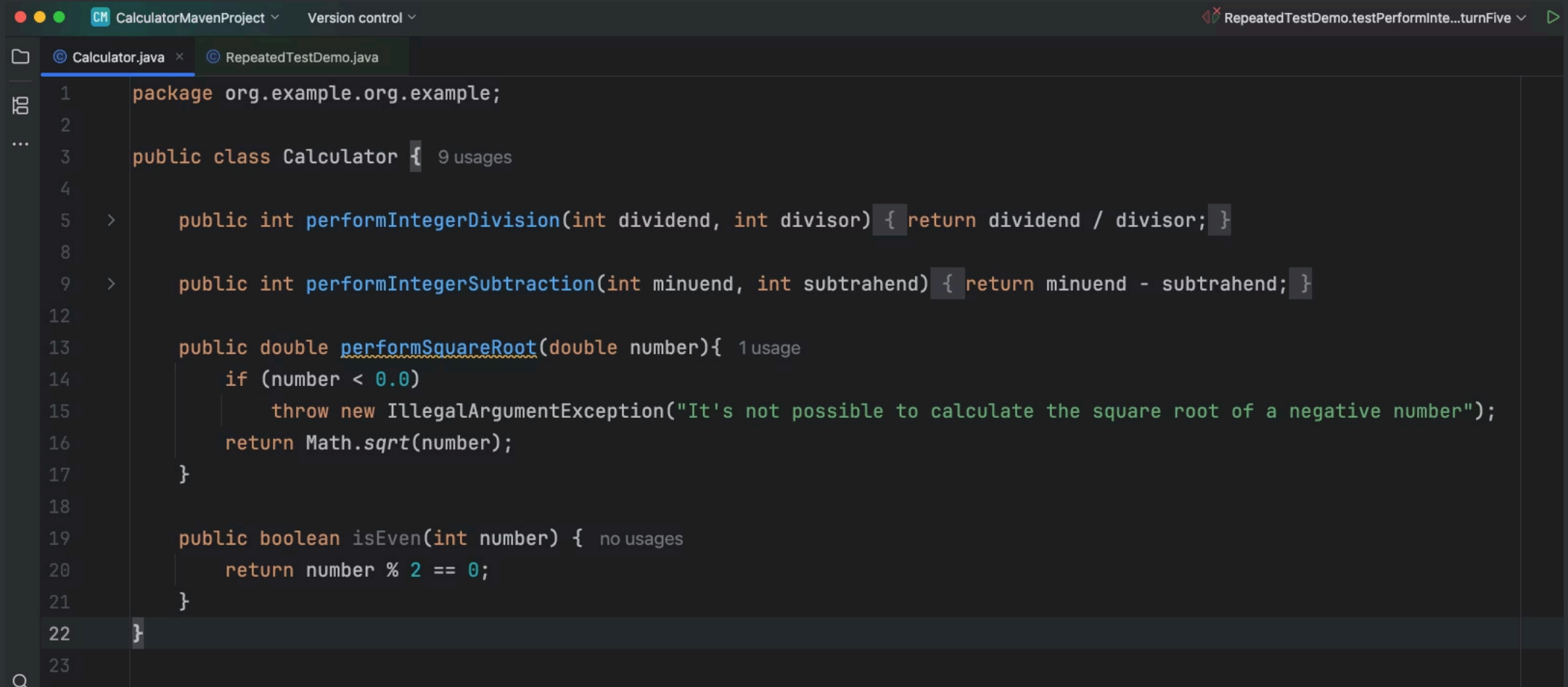
Repeated Tests

Another category is the **repeated test**, used when the **same test must run several times**.



Create a method to verify if a number is even or not

Open the `Calculator` class and create a **new method** (Lines 19-21) to verify if a number is even or not.



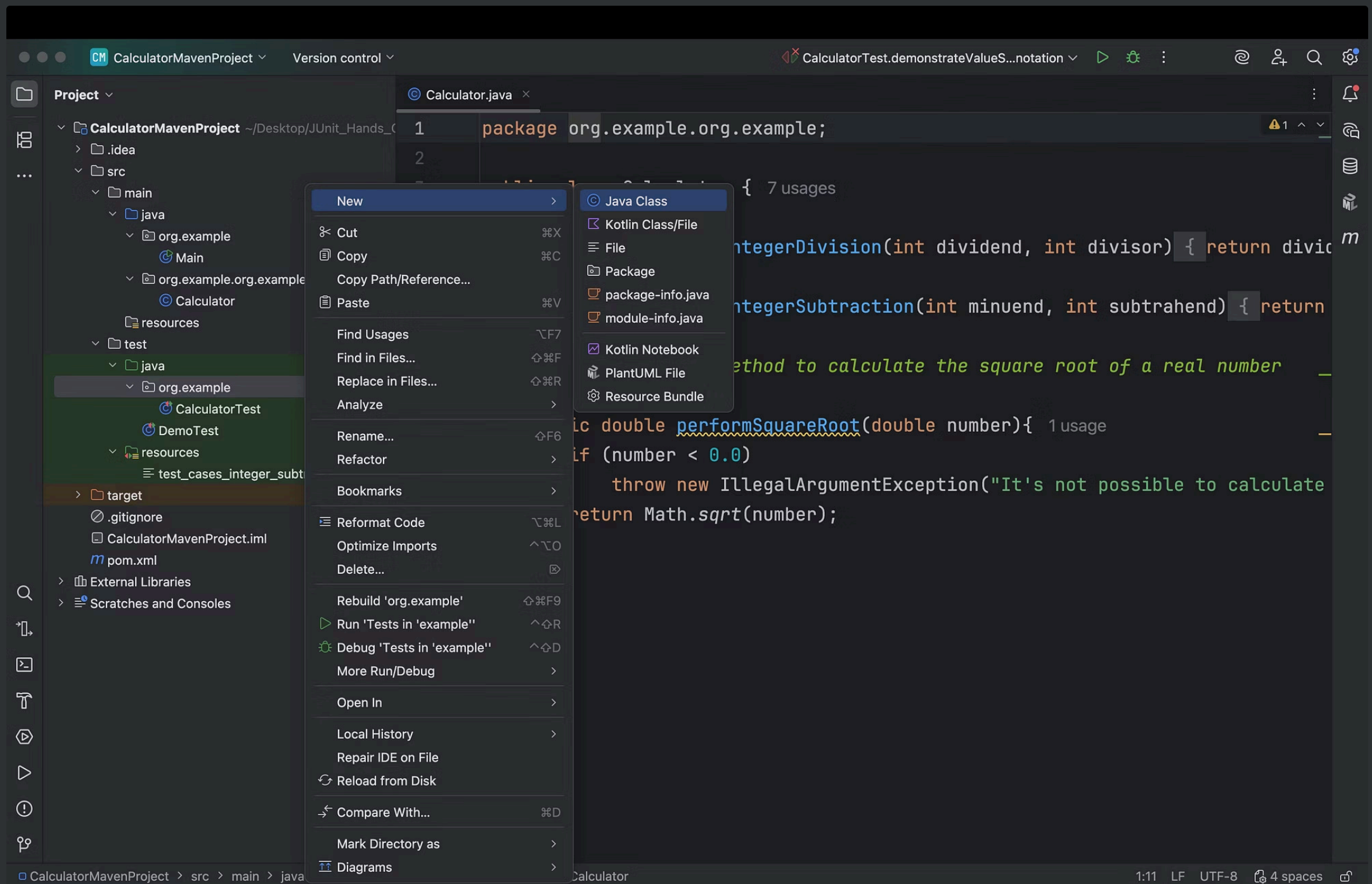
```
1 package org.example.org.example;
2
3 public class Calculator { 9 usages
4
5     > public int performIntegerDivision(int dividend, int divisor) { return dividend / divisor; }
6
7
8     > public int performIntegerSubtraction(int minuend, int subtrahend) { return minuend - subtrahend; }
9
10
11
12
13     public double performSquareRoot(double number){ 1 usage
14         if (number < 0.0)
15             throw new IllegalArgumentException("It's not possible to calculate the square root of a negative number");
16         return Math.sqrt(number);
17     }
18
19     public boolean isEven(int number) { no usages
20         return number % 2 == 0;
21     }
22 }
23
```

Repeated Tests with @RepeatedTest

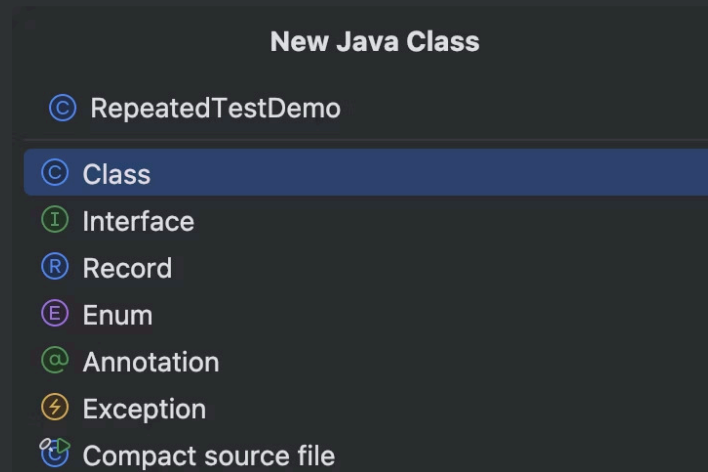
- Replace `@Test` with `@RepeatedTest(n)` to repeat a test **n times**.
- **Each repetition** behaves like an **independent test execution**.
- Lifecycle methods (`@BeforeEach`, `@AfterEach`) run **before and after every repetition**.
- Useful for verifying **stability** and **consistency across multiple runs**.



Create a new test class to practice the use of @RepeatedTest



Create a new test class to practice the use of @RepeatedTest (cont.)

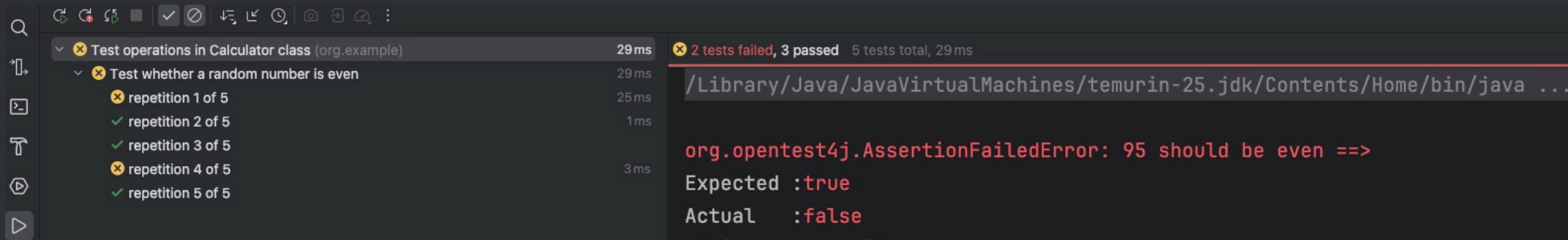


Create a new test method to practice the use of @RepeatedTest (cont.)

```
140  ⚡ @DisplayName("Test whether a random number is even")
141    @RepeatedTest(5)
142  🟢 public void testIsEven_WhenProvidedSeveralNumbers_ShouldVerifyIfEachOneIsEven() {
143      // Arrange
144      int randomNumber = (int) (Math.random() * 100);
145      // Act
146      boolean actualResult = calculator.isEven(randomNumber);
147      // Assert
148      assertTrue(actualResult, ()-> randomNumber + " should be even");
149  }
150 }
```

Run the test method

Every time you run, a different test result is shown, since the numbers are being randomly generated.



Injecting Repetition and Test Info

You can inject `RepetitionInfo` into the **test method** to access:

Current repetition number

Total number of repetitions

You can also inject `TestInfo` to retrieve:

Test method name

Display name defined with
`@DisplayName`

Both objects help customize output and test diagnostics.



Using RepetitionInfo and TestInfo

- 1 RepetitionInfo allows printing details like "1 of 5"
- 2 TestInfo lets you display the **method name** during execution
- 3 Supports debugging, logging, and dynamic behavioural checks
- 4 Each repetition is fully independent, yet carries its own metadata



Methods of RepetitionInfo object

```
140 @DisplayName("Test whether a random number is even")
141 @RepeatedTest(5)
142 @
143 System.out.println("Repetition " + repetitionInfo.~
144 // Arrange
145 int randomNumber = (int) (Math.random() * 100)
146 // Act
147 boolean actualResult = calculator.isEven(random
148 // Assert
149 assertTrue(actualResult, ()-> randomNumber + "
150 }
151 }
```

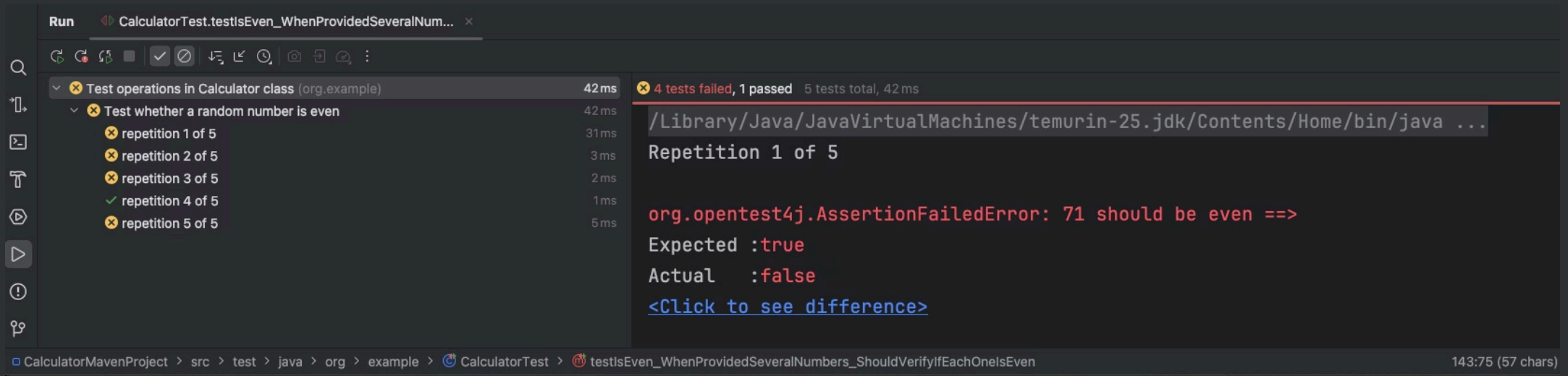
getCurrentRepetition()	int
getTotalRepetitions()	int
getFailureCount()	int
toString()	String
getFailureThreshold()	int
equals(Object obj)	boolean
hashCode()	int
getClass()	Class<? extends RepetitionInfo>

Methods of RepetitionInfo object (contd.)

You can inject `RepetitionInfo` into the **test method** to access the **current repetition number** and **total number of repetitions**

```
140     @DisplayName("Test whether a random number is even")
141     @RepeatedTest(5)
142     @Test
143     public void testIsEven_WhenProvidedSeveralNumbers_ShouldVerifyIfEachOneIsEven(RepetitionInfo repetitionInfo) {
144         System.out.println("Repetition " + repetitionInfo.getCurrentRepetition() + " of " + repetitionInfo.getTotalRepetitions() );
145         // Arrange
146         int randomNumber = (int) (Math.random() * 100);
147         // Act
148         boolean actualResult = calculator.isEven(randomNumber);
149         // Assert
150         assertTrue(actualResult, ()-> randomNumber + " should be even");
151     }
```

Run the test method



The screenshot shows the Run window of an IDE. The top bar indicates the test being run: `CalculatorTest.testIsEven_WhenProvidedSeveralNum...`. The left pane shows a tree view of the test results:

- Test operations in Calculator class (org.example) - 42 ms
 - Test whether a random number is even - 42 ms
 - repetition 1 of 5 - 31 ms (failed)
 - repetition 2 of 5 - 3 ms (failed)
 - repetition 3 of 5 - 2 ms (failed)
 - repetition 4 of 5 - 1 ms (passed)
 - repetition 5 of 5 - 5 ms (failed)

The right pane shows the output of the failed test:

```
/Library/Java/JavaVirtualMachines/temurin-25.jdk/Contents/Home/bin/java ...  
Repetition 1 of 5  
  
org.opentest4j.AssertionFailedError: 71 should be even ==>  
Expected :true  
Actual   :false  
<Click to see difference>
```

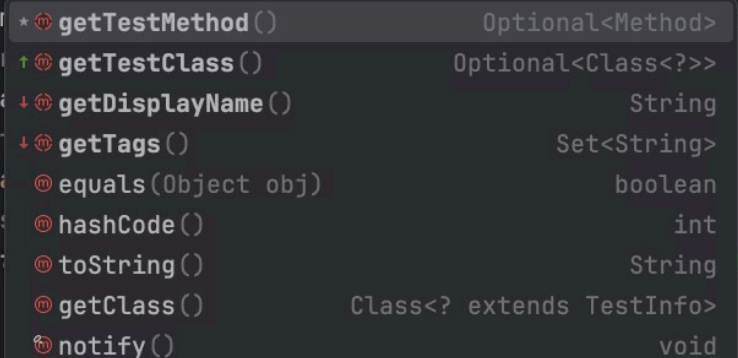
The bottom status bar shows the current file path: `CalculatorMavenProject > src > test > java > org > example > CalculatorTest > testIsEven_WhenProvidedSeveralNumbers_ShouldVerifyIfEachOnIsEven` and the cursor position: `143:75 (57 chars)`.

In this run, only one test passed.

Methods of TestInfo object

You can also inject `TestInfo` to retrieve the **Test method name** and the **Display name** defined with `@DisplayName`

```
140     @DisplayName("Test whether a random number is even")
141     @RepeatedTest(5)
142     @ @ public void testIsEven_WhenProvidedSeveralNumbers_ShouldVerifyIfEachOneIsEven(RepetitionInfo repetitionInfo, TestInfo testInfo) {
143         testInfo.
144         System.out.println("Current repetition: " + repetitionInfo.getCurrentRepetition() + " of " + repetitionInfo.getTotalRepetitions() );
145         // Assert that the number is even
146         int random = RandomUtil.getRandomNumber();
147         // Assert that the number is even
148         boolean isEven = random % 2 == 0;
149         // Assert that the number is even
150         assertEquals("The number is not even", isEven, true);
151     }
152 }
```



The image shows a code editor with a tooltip for the `TestInfo` class. The tooltip lists the following methods:

- `getTestMethod()` Optional<Method>
- `getTestClass()` Optional<Class?>
- `getDisplayName()` String
- `getTags()` Set<String>
- `equals(Object obj)` boolean
- `hashCode()` int
- `toString()` String
- `getClass()` Class<? extends TestInfo>
- `notify()` void

Methods of TestInfo object (contd.)

```
140     @DisplayName("Test whether a random number is even")
141     @RepeatedTest(5)
142     public void testIsEven_WhenProvidedSeveralNumbers_ShouldVerifyIfEachOneIsEven(RepetitionInfo repetitionInfo, TestInfo testInfo) {
143         System.out.println("Running " + testInfo.getTestMethod().get().getName() );
144         System.out.println("Repetition " + repetitionInfo.getCurrentRepetition() + " of " + repetitionInfo.getTotalRepetitions() );
145         // Arrange
146         int randomNumber = (int) (Math.random() * 100);
147         // Act
148         boolean actualResult = calculator.isEven(randomNumber);
149         // Assert
150         assertTrue(actualResult, ()-> randomNumber + " should be even");
151     }
152 }
```

Run the test method

Run CalculatorTest.testIsEven_WhenProvidedSeveralNum... x

Test operations in Calculator class (org.example) 37 ms

- Test whether a random number is even 37 ms
 - repetition 1 of 5 33 ms
 - repetition 2 of 5 1 ms
 - repetition 3 of 5 1 ms
 - repetition 4 of 5 1 ms
 - repetition 5 of 5 1 ms

1 test failed, 4 passed 5 tests total, 37 ms

```
/Library/Java/JavaVirtualMachines/temurin-25.jdk/Contents/Home/bin/java ...  
Running testIsEven_WhenProvidedSeveralNumbers_ShouldVerifyIfEachOneIsEven  
Repetition 1 of 5  
  
org.opentest4j.AssertionFailedError: 29 should be even ==>  
Expected :true  
Actual   :false  
<Click to see difference>
```

In this run, 4 tests passed (See the message printed in the console - highlighted in blue - for the output produced by the `getName()` method)

```
System.out.println("Running " + testInfo.getTestMethod().get().getName());
```

Customizing Display Names with @RepeatedTest

@RepeatedTest supports properties such as:

- `value = n` → number of repetitions.
- `name = "..."` → custom display name for each repetition.

Template variables available:

- `{displayName}`
- `{currentRepetition}`
- `{totalRepetitions}`

Helps create descriptive names like: ***"Test whether a random number is even – repetition 2 of 5"***



Customizing Display Names with @RepeatedTest (contd.)

Replace

```
@RepeatedTest(5)
```

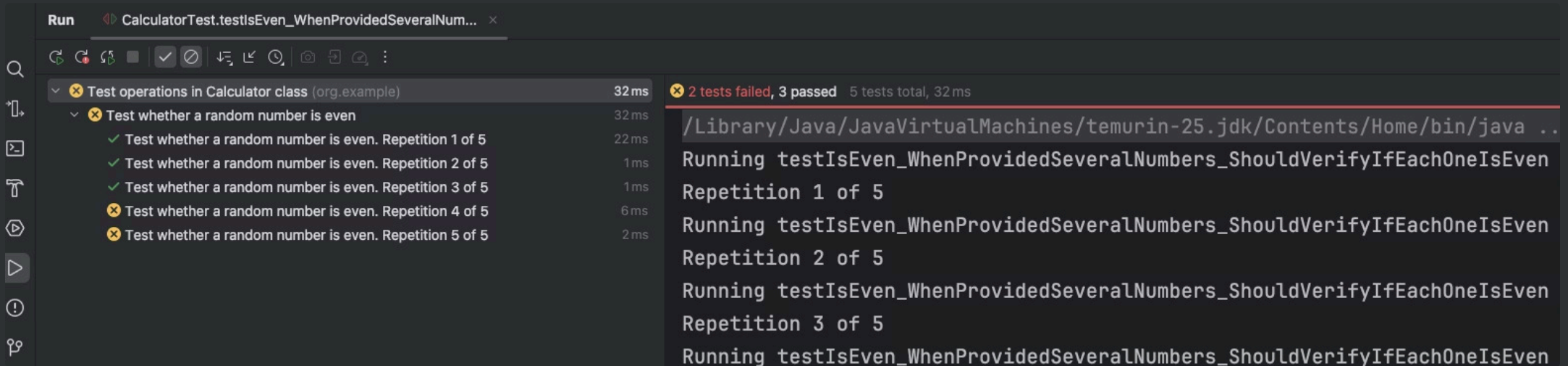
With

```
@RepeatedTest(value=5, name="{displayName}. Repetition {currentRepetition} of {totalRepetitions}")
```

```
140     @DisplayName("Test whether a random number is even")
141     @RepeatedTest(value=5, name="{displayName}. Repetition {currentRepetition} of {totalRepetitions}")
142     @
143     public void testIsEven_WhenProvidedSeveralNumbers_ShouldVerifyIfEachOneIsEven(RepetitionInfo repetitionInfo, TestInfo testInfo) {
144         System.out.println("Running " + testInfo.getTestMethod().get().getName());
145         System.out.println("Repetition " + repetitionInfo.getCurrentRepetition() + " of " + repetitionInfo.getTotalRepetitions() );
146         // Arrange
147         int randomNumber = (int) (Math.random() * 100);
148         // Act
149         boolean actualResult = calculator.isEven(randomNumber);
150         // Assert
151         assertTrue(actualResult, ()-> randomNumber + " should be even");
152     }
```

{displayName}, {currentRepetition}, and {totalRepetitions} are **template variables**.

Run the test method



The screenshot shows the 'Run' window of an IDE. The title bar indicates the test being run is 'CalculatorTest.testIsEven_WhenProvidedSeveralNum...'. The window is divided into two main sections. The left section displays a tree view of the test results:

- Test operations in Calculator class (org.example) [32 ms]
 - Test whether a random number is even [32 ms]
 - Test whether a random number is even. Repetition 1 of 5 [22 ms] (passed)
 - Test whether a random number is even. Repetition 2 of 5 [1 ms] (passed)
 - Test whether a random number is even. Repetition 3 of 5 [1 ms] (passed)
 - Test whether a random number is even. Repetition 4 of 5 [6 ms] (failed)
 - Test whether a random number is even. Repetition 5 of 5 [2 ms] (failed)

The right section shows the command prompt output, which is the execution of the test method:

```
/Library/Java/JavaVirtualMachines/temurin-25.jdk/Contents/Home/bin/java ..  
Running testIsEven_WhenProvidedSeveralNumbers_ShouldVerifyIfEachOneIsEven  
Repetition 1 of 5  
Running testIsEven_WhenProvidedSeveralNumbers_ShouldVerifyIfEachOneIsEven  
Repetition 2 of 5  
Running testIsEven_WhenProvidedSeveralNumbers_ShouldVerifyIfEachOneIsEven  
Repetition 3 of 5  
Running testIsEven_WhenProvidedSeveralNumbers_ShouldVerifyIfEachOneIsEven
```

In this run, three tests passed.

Conclusion

01

Repeated tests verify **consistent behavior** across **multiple executions**

02

They combine seamlessly with **lifecycle methods** and **injected metadata**

03

Custom repetition names improve clarity in test reports

04

Ideal for **stress-testing small units** or **validating deterministic behavior**

Open Floor Discussion

- When should **repeated tests** be **preferred** over **parameterized tests**?
- How could **repeated tests** expose **flaky** or **state-dependent issues**?
- What scenarios in your **codebase** could benefit from **repeated execution**?

